Secure Login – Sprechskript

Zum Ablesen · Gesamtdauer ca. 9–10 Minuten · Person A ca. 4,5 Min., Person B ca. 5 Min. · Kursiv = Regieanweisung

Vorbereitung (vor dem Vortrag)

- Im Projektordner `docker compose up -d --build` starten, danach **http://127.0.0.1:8081** im Browser öffnen.
- In `.env` sind SMTP_USER, SMTP_PASSWORD und SMTP_FROM gesetzt, damit die Mail wirklich ankommt. Postfach der Demo-Adresse offen halten.
- Authenticator-App auf dem Handy bereit. Zum Zurücksetzen vor dem Vortrag: `docker compose down -v`, dann wieder starten.
- Zweites Fenster mit dem Code bereithalten (`src/`-Ordner und `public/login.php`), falls Fragen zum Aufbau kommen.
- Notfall ohne Internet: SMTP_PASSWORD leer lassen, dann erscheint der Code direkt in der Seite.

TEIL 1 · PERSON A – Idee, Aufbau, Anmeldevarianten

1. Begrüßung und Idee (ca. 1 Min.)

Guten Tag, wir stellen Ihnen unser Projekt „Secure Login“ vor. Das ist ein Anmeldesystem mit Registrierung, Login, Passwort-Zurücksetzen und optionaler Zwei-Faktor-Absicherung.

Der Schwerpunkt liegt auf Sicherheit. Passwörter werden nie im Klartext gespeichert, jedes Formular ist gegen Fälschungen geschützt, und wiederholte Versuche werden begrenzt. Außerdem kann jeder Benutzer selbst entscheiden, wie stark sein Konto abgesichert sein soll.

2. Technischer Aufbau (ca. 1,5 Min.)

Die Anwendung besteht aus zwei Docker-Containern: einem mit Apache und PHP, einem mit der Datenbank MariaDB. Alles startet mit einem einzigen Befehl, *docker compose up*.

Der Code ist bewusst einfach gehalten. In *src* liegen 13 kleine Klassen mit etwa 1.200 Zeilen, jede mit genau einer Aufgabe: zum Beispiel *Auth* für den Login-Ablauf, *EmailCodes* für die Mail-Codes, *TwoFactor* und *Totp* für die App, *Security* für Passwörter und Verschlüsselung.

Der Ordner *public* enthält nur die Seiten, die der Browser aufruft, jeweils kurze Dateien. Alles andere, also Logik, Vorlagen und Konfiguration, liegt außerhalb und ist von außen nicht erreichbar. Als einzige Bibliothek nutzen wir PHPMailer für den Mailversand.

Regie: kurz den Ordnerbaum zeigen, src und public markieren.

3. Die drei Anmeldevarianten (ca. 2 Min.)

Variante 1: nur Passwort. E-Mail und Passwort, danach ist man eingeloggt. Das Passwort muss mindestens zwölf Zeichen haben.

Variante 2: Code per E-Mail. Nach dem Passwort bekommt man einen sechsstelligen Code ins Postfach. Er ist zehn Minuten gültig, und man darf ihn fünfmal falsch eingeben, danach ist er ungültig. Bevor man das einschalten kann, wird die Adresse durch einen Code bestätigt. So kann niemand eine fremde Adresse eintragen.

Variante 3: Authenticator-App. Man scannt einen QR-Code mit einer App wie Google Authenticator. Danach braucht jeder Login den Code aus der App. Er ändert sich alle 30 Sekunden und funktioniert ohne Internet. Grundlage ist der Standard RFC 6238, den wir selbst umgesetzt und mit dem offiziellen Testvektor geprüft haben.

Man kann Variante 2 und 3 auch gleichzeitig einschalten.

Übergabe: „Wie ein Login technisch abläuft und wie wir die Daten schützen, erklärt jetzt [Name Person B].“

TEIL 2 · PERSON B – Ablauf, Sicherheit, Demo, Fazit

4. Ablauf eines Logins (ca. 1 Min.)

Beim Login prüft der Server zuerst das Passwort gegen den gespeicherten Hash. Stimmt es, entscheidet das Konto, wie es weitergeht: Hat es die E-Mail-Bestätigung eingeschaltet, kommt die Code-Abfrage, hat es die App eingerichtet, kommt der App-Code.

Bis alle Schritte bestanden sind, ist man nicht eingeloggt. Das Konto steht dann nur als „wartend“ in der Sitzung. Erst am Ende bekommt man eine neue Sitzungs-ID und Zugriff auf die Konto-Seite.

5. Sicherheit (ca. 2 Min.)

Passwörter: Wir speichern nur einen bcrypt-Hash mit zufälligem Salt. Aus der Datenbank lässt sich kein Passwort zurückrechnen.

Codes in der Datenbank: Mail-Codes liegen nur als HMAC, Reset-Links nur als Hash, und das App-Geheimnis ist mit libsodium verschlüsselt. Ein Datenbank-Diebstahl liefert also keine verwendbaren Codes.

Brute-Force: Wir begrenzen die Versuche in der Datenbank, zum Beispiel 20 Logins in 15 Minuten pro IP. Ein App-Code gilt außerdem nur einmal, auch wenn er noch zeitlich gültig wäre.

Angriffe auf die Seite: Jedes Formular hat ein CSRF-Token, alle Datenbankabfragen sind Prepared Statements, und jede Ausgabe wird gegen Schadcode entschärft. Das Session-Cookie ist HttpOnly und SameSite, und eine Content-Security-Policy verbietet Inline-Skripte.

Passwort vergessen: Der Link ist 30 Minuten gültig und lässt sich nur einmal einlösen. Die Seite antwortet immer gleich, damit niemand herausfinden kann, welche Adressen registriert sind.

Regie: bei Bedarf in schema.sql oder Security.php zeigen.

6. Live-Demo (ca. 1,5 Min.)

1) Registrieren mit einer E-Mail und einem Passwort ab zwölf Zeichen, danach einloggen.

2) Auf der Konto-Seite die E-Mail-Bestätigung aktivieren. Wir bekommen eine Mail mit einem Code. Regie: Mail im Postfach zeigen, Code eingeben.

3) Ausloggen und wieder einloggen. Jetzt fragt der Login nach dem Code aus der Mail. Regie: neue Mail zeigen, Code eingeben.

4) Optional: die Authenticator-App einrichten, QR-Code scannen, Code eingeben. Beim nächsten Login braucht man beide Codes.

Regie: Falls die Mail nicht ankommt, ruhig bleiben. Im Entwicklungsmodus steht der Code direkt in der Seite, oder im Log: docker compose logs app.

7. Fazit (ca. 0,5 Min.)

Wir haben ein vollständiges Anmeldesystem gebaut, bei dem der Benutzer die Sicherheitsstufe selbst wählt. Wir haben den Code bewusst klein und lesbar gehalten und alle Abläufe im laufenden System durchgetestet. Mögliche nächste Schritte wären Wiederherstellungscodes für die App und mehr automatische Tests. Vielen Dank, wir beantworten gern Ihre Fragen.

Mögliche Fragen und kurze Antworten

Frage	Antwort
Warum bcrypt und nicht SHA-256?	SHA-256 ist schnell und damit leicht zu erraten. bcrypt ist absichtlich langsam und enthält einen Salt.
Was passiert, wenn das Handy weg ist?	Dann kommt man mit der App nicht mehr hinein. Eine Wiederherstellung gibt es noch nicht, die 2FA müsste in der Datenbank entfernt werden. Das wäre eine Erweiterung.
Warum liegt src außerhalb von public?	Nur public ist über den Browser erreichbar. So kann niemand den Quellcode oder die Konfiguration direkt aufrufen.
Warum hat jedes Formular ein CSRF-Token?	Damit eine fremde Seite im Namen des Benutzers keine Aktion auslösen kann.
Wozu der APP_KEY?	Er verschlüsselt die App-Geheimnisse und salzt die Mail-Codes. Er darf sich nach dem Start nicht ändern.
Warum nur eine Bibliothek?	Wir wollten wenig Abhängigkeiten. Nur der Mailversand nutzt PHPMailer, weil eine eigene SMTP-Anbindung fehleranfälliger wäre.
Wie wurde getestet?	16 automatische Tests der Bausteine (u. a. RFC-Testvektor) und ein kompletter Durchlauf im laufenden System: Registrierung, Login, Codes, Sperre nach Fehlversuchen, Reset.